Course Title	Introduction to Python Programming.	Semester	I
Course Code	25PL104	Course Type	PLC/Theory+Lab
Total No. of Contact Hours	25hrs(Th)+20(Pr)= 45 hrs	CIA Marks	50
No. of Contact Hours/week (L:T:P)	2:0:2	SEE Marks	50
Credits	3	Total Marks	100
Teaching Department	CSE Cluster	Exam. Duration	3 hrs (Theory) + 3 hrs (Practical)

Course Overview: This course, Introduction to Python Programming, helps students learn the basics of Python through both theory and practical sessions. It covers key programming topics like variables, data types, loops, functions, and error handling. Students will also work with data structures such as lists, tuples, and dictionaries, and learn file handling, string operations, and basic debugging. The course includes object-oriented programming using classes and objects. Hands-on activities and mini-projects support learning, and by the end, students will be ready to use Python for solving simple real-world problems and move on to advanced topics like data science and machine learning.

Course objectives: The student will be able to:

1. Identify**,** demonstrate, and apply core concepts of Python such as data types, variables, control flow, functions, and basic error handling to build a strong programming foundation.
2. Select**,** utilize, and compare essential data structures like lists, tuples, and dictionaries to create efficient data-handling solutions.
3. Apply**,** use**,** and revise string manipulation, file and folder management, and debugging techniques to prepare and develop reliable Python programs.
4. Design**,** create**,** and evaluate object-oriented programs by utilizing classes and objects and applying key OOP principles such as encapsulation and polymorphism to develop reusable code.

PREREQUISITES: This course requires students to have basic familiarity with using computers and an understanding of fundamental mathematical concepts such as arithmetic and logical operations. Strong logical thinking and problem-solving skills will further support effective learning of Python programming.

Module-1 Introduction to Python	**5 Hours**

Python Basics: Entering Expressions into the Interactive Shell, The Integer, Floating-Point, and String Data Types, String Concatenation and Replication, Storing Values in Variables, Your First Program, Dissecting Your Program, Flow control: Boolean Values, Comparison Operators, Boolean Operators, Mixing Boolean and Comparison Operators, Elements of Flow Control, Program Execution, Flow Control Statements, Importing Modules, Ending a Program Early with sys.exit(), Functions: def Statements with Parameters, Return Values and return Statements, The None Value, Keyword Arguments and print(), Local and Global Scope, The global Statement, Exception Handling.

Activity: Guess the Number.
Write a Python program that randomly picks a number between 1 and 50. Ask the user to guess the number, giving them only 6 chances. After each guess, tell them if their guess is too high, too low, or correct. If they guess correctly, show how many attempts they took. If not, reveal the correct number after 6 tries.

Textbook 1: Chapters 1 – 3

Module-2:Lists and Dictionary	**5 Hours**

Lists: The List Data Type, Working with Lists, Augmented Assignment Operators, Methods, Example Program: Magic 8 Ball with a List, List-like Types: Strings and Tuples, References, Dictionaries and Structuring Data: The Dictionary Data Type, Pretty Printing, Using Data Structures to Model Real-World Things.

Activity: Fortune Teller Program
Write a Python program that acts like a fortune teller. Make a list of at least 10 fortunes (like "You will have a great day" or "Be careful today"). Ask the user to enter their name and a yes/no question. Randomly choose a fortune and show it as the answer. Save the user's name and their fortune in a dictionary. Repeat this for 3 users and show all the names with their fortunes at the end.

Textbook 1: Chapters 4 – 5

Module-3 Strings	**5 Hours**

Manipulating Strings: Working with Strings, Useful String Methods, Project: Password Locker, Project: Adding Bullets to Wiki Markup Reading and Writing Files: Files and File Paths, The os.path Module, The File Reading/Writing Process, Saving Variables with the shelve Module, Saving Variables with the print.format() Function, Project: Multi clipboard.

Activity: Random Quiz Generator
Create a Python program that stores 10 Indian state-capital pairs in a dictionary. Randomly pick 5 states and write quiz questions like "What is the capital of [State]?" to a file named quiz.txt, and the correct answers to answers.txt, line by line.

Textbook 1: Chapters 6 , 8

Module-4: Organizing Files	**5 Hours**

Organizing Files: The shutil Module, walking a Directory Tree, Compressing Files with the zipfile Module, Backing Up a Folder into a ZIP File, debugging: Raising Exceptions, Getting the Traceback as a String, Assertions, Logging, IDLE"s Debugger.

Activity: Folder Backup and Debugging
Create a Python program to back up a folder into a ZIP file using the zipfile module. Use os, shutil, and logging to walk through the folder, handle errors, and log exceptions. Add assertions to check if the folder exists.

Textbook 1: Chapters 9-10

Module-5 Classes and Objects	**5 Hours**

Classes and objects: Programmer-defined types, Attributes, Rectangles, Instances as return values, Objects are mutable, Copying, Classes and functions: Time, Pure functions, Modifiers, Prototyping versus planning, Classes and methods: Object-oriented features, Printing objects, Another example, A more complicated example, The init method, The _str_ method, Operator overloading, Type-based dispatch, Polymorphism, Interface and implementation.

Activity: Library Book System
Create a Book class with attributes like title, author, and availability. Add methods to issue and return a book. Use a dictionary to track issued books. Demonstrate changing availability through methods.

Textbook 2: Chapters 15 – 17

Practice Experiments: (20 Hours)

Program 1:

a) Title: Design and develop a Python program to read student details and compute total marks and percentage.

Problem Description: The program should prompt the user to input a student's name, USN, and marks in three subjects. It should then calculate the total and percentage, and display all the details with appropriate messages.

Method: Use input functions to collect student details, apply arithmetic operations to compute total and percentage, and display the results using formatted output.

b) Title: Design and develop a Python program to determine if a person is a senior citizen.

Problem Description: The program should read the name and year of birth of a person and determine whether they are a senior citizen(age≥60).

Method: Use input functions to collect personal details, calculate age by subtracting the year of birth from the current year, and apply conditional statements to check and display if the person is a senior citizen.

Theory Reference: Module 1

Program 2:

a) Title: Generate a Fibonacci sequence up to N terms.

Problem Description: The program reads a value N and generates the first N terms of the Fibonacci series.
Method: Define a function to generate Fibonacci numbers, and use a loop to iteratively compute and display the sequence up to N terms.

b) Title: Write a function to calculate the factorial of a number and compute binomial coefficient.
Problem Description: Implement a function to calculate the factorial of a number and use it to compute the binomial coefficient given N and R.

Method: Define a function using either recursion or iteration to compute factorial values, then apply the binomial coefficient formula.
Theory Reference: Module 1

Program 3:
Title: Calculate mean, variance, and standard deviation for N numbers.
Problem Description: Read N numbers into a list and compute the mean, variance, and standard deviation.
Method: Use a list to store N numbers, apply loops and arithmetic operations to compute the mean and variance, and use the math module to calculate the standard deviation.
Theory Reference: Module 1

Program 4:
Title: Determine frequency of each digit in a multi-digit number.

Problem Description: Read a multi-digit number as a string and display how many times each digit (0–9) appears.

Method: Read the number as a string, use a dictionary to track digit frequencies, and apply loops to iterate through each character and update the count accordingly.

Theory Reference: Module 1

Program 5:

Title: Identify 10 most frequent words in a text file.

Problem Description: Read a text file, compute the frequency of each word, and display the top 10 most frequent words using a dictionary

Method: Use file handling to read the text, store word frequencies in a dictionary, and apply sorting to identify and display the top 10 most frequent words.

Theory Reference: Module 2

Program 6:

Title: Sort contents of a text file and write to a new file.

Problem Description: Read a text file, sort its lines or words, and write the sorted contents to another text file.

Method: Read the contents of the input file into a list, apply built-in sorting functions to sort the lines or words, and use file handling to write the sorted output to a new file.

Theory Reference: Module 2

Program 7:

Title: Back up a folder into a ZIP file using Python.

Problem Description: Create a Python program to back up all contents of a given folder into a ZIP file.

Method: Use the os module to navigate the folder structure, shutil or zipfile module to compress files, and automate the process of archiving all contents into a ZIP file.

Theory Reference: Module 3

Program 8:

Title: Division with assertion and exception handling.

Problem Description: Define a function DivExp(a, b) that returns a/b, uses assertion for a > 0, and raises an exception if b == 0. The main program reads two numbers and calls this function.

Method: Define a function to perform division with an assertion to ensure the numerator is positive, include exception handling to catch division by zero, and call the function from the main program with user input.

Theory Reference: Module 4

Program 9:

Title: Perform addition of complex numbers using objects.

Problem Description: Create a class Complex, define methods to add complex numbers, and compute the sum of N complex numbers provided by the user.

Method: Use object-oriented programming by defining a Complex class with attributes for real and imaginary parts, implement a method to add complex numbers, and create objects to input and compute the sum of N complex numbers.

Theory Reference: Module 5

Program 10:

Title: Student scorecard using class and list.

Problem Description: Design a Student class with methods to read marks of three subjects and compute total and percentage. Use lists to manage marks.

Method: Implement a Student class using object-oriented principles with an _init_() method to initialize data, use

lists to store marks, and define class methods to read marks, calculate total and percentage, and display the results.
Theory Reference: Module 5

Suggested Learning Resources:
Textbooks:

Al Sweigart,"Automate the Boring Stuff with Python",1 stEdition, No Starch Press, 2015.

Allen B. Downey, "Think Python: How to Think Like a Computer Scientist", 2 nd Edition, Green Tea Press, 2015.

Reference Books:

Introduction to Python Programming, apart from the ones already mentioned, is **"Python Crash Course" by Eric Matthes (2nd Edition, No Starch Press, 2019)**.

Web links and Video Lectures (e-Resources):

1. https://www.learnbyexample.org/python/
2. https://www.learnpython.org/
3. https://pythontutor.com/visualize.html#mode=edit

Activity-Based Learning (Suggested Activities in Class)/ Practical Based learning

1. Flipped Class Room.
2. Multiple Choice Questions.
3. Mini Projects.

Course outcomes: At the end of the course, the student will be able to

Define and explain the fundamental concepts of Python programming.

CO1	Apply knowledge of Python programming constructs in solving simple computational problems.
CO2	Analyze and identify the output for the given python code snippet.
CO3	Develop structured programs using various programming techniques.
CO4	Use engineering tools for developing efficient programming solutions.
CO5	Collaborate effectively as an individual or team member to communicate and present programming applications.

CO-PO Mapping:

CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	-	-	-	-	-	-	-	-	-	-
CO2	-	3	-	-	-	-	-	-	-	-	-
CO3	-	-	1	-	-	-	-	-	-	-	-
CO4	-	-	-	-	3	-	-	-	-	-	-
CO5	-	-	-	-	-	-	-	2	2	-	-

High-3, Medium-2, Low-1